Light Link

Homey Pro app specification — a universal remote-to-light mapping layer

Version 2.0 **Date** 8 August 2026 **Target** Homey Pro, Apps SDK v3, TypeScript **Status** Implementation-ready

CONTENTS

1 Product definition
2 Platform constraints
3 Blocking spike — read first
4 Architecture
5 Source discovery and normalisation
6 Managed flow bridge
7 Light control runtime
8 User experience
9 Persistence, health and repair
10 Codebase structure and environment
11 Testing
12 Security and robustness
13 Acceptance criteria
14 Implementation plan
15 Decisions locked
16 Platform references

1. Product definition

Light Link lets a user repurpose any already-paired remote, switch, button panel or rotary controller as an intuitive controller for one or more already-paired lights, replacing a collection of hand-authored Homey Flows with a single mapping.

Mental model: *this remote controls these lights, and this physical action performs this lighting function.*

Step	User action	App responsibility
1. Remote	Select one paired remote or switch.	Discover usable input events via capabilities and flow trigger cards; normalise into a selectable catalogue.
2. Lights	Select one or more paired lights, or a zone.	Compute the common and partial capability matrix.
3. Controls	For each light function, choose a remote event.	Validate conflicts; compile logical bindings; generate the required bridge flows.
4. Save	Save the controller.	Create the virtual device, start listeners, reconcile managed flows, begin control.

1.1 Principles

- No manual flow authoring for ordinary use.
- Function-first: users choose which event drives Toggle, Brighter, Dimmer, Warmer, Colder — not the reverse.
- Multiple target lights are first-class from release one.
- Only events actually exposed by the selected integration are offered. Unavailable gestures are never invented.
- Raw card IDs, Matter endpoints, tokens and vendor values stay hidden outside diagnostics.
- Known-device recipes improve labels and defaults; generic discovery remains the base compatibility mechanism.
- **Keep complexity behind the selectors.** The user-facing model never grows beyond: pick remote → pick lights → assign event to function → save.

1.2 Non-goals for MVP

- Pairing the physical remotes or lights.
- Replacing vendor apps, or editing configuration held inside a vendor ecosystem such as the Hue app.
- Colour (hue/saturation) mapping, scene orchestration, conditional logic, user-authored sequences or macros.
- Per-action target overrides in the UI (schema only — §4.2).
- Derived double-click. It delays single-press execution and is not worth the cost at MVP.
- Homey Cloud. The design depends on broad local Web API access.

2. Platform constraints

2.1 Remote input is exposed two ways

Path	Mechanism	Status
Capability	Subscribe to a capability change on the source device via the Web API.	Rare. **No** reference device uses it. Adapter interface in MVP, implementation deferred.
Managed flow bridge	The integration fires a device flow trigger. Only a Flow can observe it, so the app generates one that calls an internal bridge action card.	**Primary path.** All four reference devices.

> COMPATIBILITY BOUNDARY — STATE THIS PLAINLY IN APP STORE COPY
>
> "Supports any device" means any already-paired source for which Homey exposes a usable capability change, or a flow trigger card bindable to that device. If the owning integration exposes neither, the input is unobservable through public Homey interfaces and no app can reach it.

2.2 Argument enumerability

Argument type	Enumerable	Handling
None	Yes	One binding, one flow.
`dropdown`	Yes — `values[]` carry id and localised label	Expand to one selectable event per value. The common case.
Numeric / range	Yes, but must not be exposed	Collapse to one selectable event; range-expand at compile time (§6.4).
`autocomplete`	No	Excluded from the catalogue. Manual mode only.

2.3 Reference-device behaviour

These four are fixtures for validating that the generic model covers common integration patterns — not a hardcoded supported-device list.

Device	Path	Observed behaviour	Required handling
IKEA STYRBAR E2001/E2002	Zigbee, local	Four controls with press and long-press triggers. Fast. Firmware quirk: holding left or right emits a short-press event *before* the hold. Release is not emitted for every control.	Supersede window (§7.6). Ramp safety timeout (§7.7). Never invent a release.
Hue Dimmer v2 RWL022	Hue Bridge	Well-behaved. Button-pressed trigger with the button as a dropdown argument.	Reference implementation. Expand button values into normalised press events. Do not offer hold if the integration does not expose it.
Hue Tap Dial RDM002	Hue Bridge	Four buttons plus dial. Rotation arrives as discrete press-like events, not a continuous stream; magnitude may be tokenised. Hold is not exposed as it is in the Hue app.	Collapse dial variants to left/right. Preserve magnitude for brightness scaling. Stepping — not ramping — is the primary dimming path.
IKEA BILRESA scroll wheel	Matter/Thread	Presses are not capabilities. Wheel movement may be encoded as button/count arguments and changes across integration versions. **Trigger cards disappear after a Homey restart**, requiring device re-add. Fast multi-notch turns drop events; devices sleep and lose connection.	Runtime discovery only, never a static recipe. Range-expanded flows. One-tap re-attach (§9.4). Acceleration off by default.

2.4 Pairing-path sensitivity

Compatibility is determined by the integration and its exposed event surface, not the hardware model name. The same Hue device exposes a richer surface through the Philips Hue app than through a Matter path. Known-device recipes must therefore match on **owner app + driver + event-surface fingerprint**, never on model name alone.

3. Blocking spike

> RESOLVE BEFORE ANY OTHER WORK
>
> The Web API's `ManagerFlow` exposes `createFlow`, `updateFlow`, `deleteFlow` and `getFlowCardTriggers`, and apps holding `homey:manager:api` can reach the Web API through `HomeyAPI.createAppAPI`. Athom's documentation says apps may use *some* endpoints — it does not confirm that flow **writes** are among them rather than reads only. The entire bridge in §6 depends on this. Do not begin Phase 1 without an answer in writing.

3.1 Phase 0 exit criteria

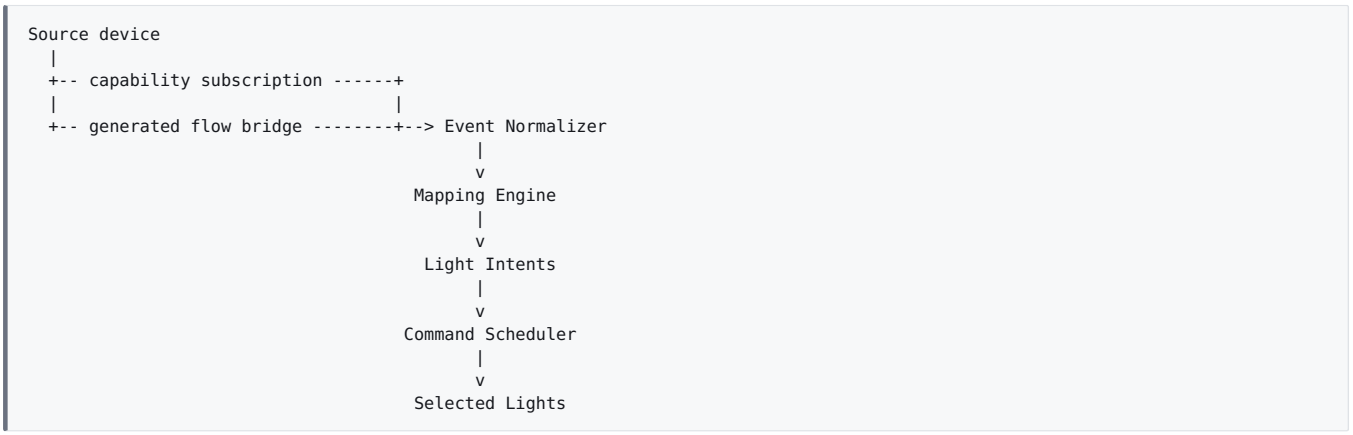
1. Create an in-app `HomeyAPI` instance and enumerate devices.
2. Enumerate flow trigger cards and resolve a selected source device against card arguments.
3. **Create a flow that calls a bridge action card, then delete it.**
4. Forward a numeric trigger token through the numeric bridge card.
5. Control one light via `setCapabilityValue` and observe target capability updates.
6. Measure press-to-light latency on Zigbee-local, Hue Bridge and Matter/Thread.

3.2 Fallback ladder if writes are refused

1. **Settings-page provisioning.** Perform flow writes from the app's settings page under the user's own authenticated session rather than the app context.
2. **Guided manual mode.** The app emits a per-binding instruction and detects when the resulting flow appears. Slower to set up; mapping model, engine, targets and UI all unchanged.

Note that `homey:manager:api` is deliberately broad. Local CLI installation is unaffected; App Store submission must explain the permission, since the app discovers and controls other paired devices and manages compatibility flows on the user's behalf.

4. Architecture



Component	Responsibility
`HomeyApiService`	Owns the in-app HomeyAPI instance; exposes managers; handles reconnect.
`DeviceCatalog`	Enumerates devices, zones, owning apps and drivers, and target capabilities.
`SourceDiscoveryService`	Finds direct capabilities and bindable flow trigger cards for a source.
`SourceAdapterRegistry`	Instantiates capability, managed-flow and known-recipe adapters.
`EventNormalizer`	Translates raw vendor events into the stable input contract; performs magnitude collapse.
`MappingEngine`	Resolves normalised events into configured light intents.
`SupersedeGate`	Resolves the press-versus-hold race on controls carrying both (§7.6).
`CommandScheduler`	Coalesces bursts, serialises per-target writes, maintains optimistic desired state.
`LightTargetAdapter`	Reads capability metadata; executes intents against targets.
`TargetResolver`	Resolves device lists and zones into a concrete target set plus capability matrix.
`FlowBridgeManager`	Compiles logical bindings into generated flows; reconciles their lifecycle.
`ControllerRuntime`	Owns all listeners and state for one virtual controller device.
`HealthMonitor`	Detects missing sources and targets, broken flows, changed card schemas.

4.1 Virtual controller device

Each configuration becomes one app-owned virtual device, which is the lifecycle boundary for the relationship: source reference, targets, mappings, managed flow references, runtime status, diagnostics. Creating one runs a custom pairing flow; editing reuses a repair session with existing values preselected. Use device class `service` unless testing shows `remote` gives materially better Homey UX.

4.2 Responsibility boundaries

SDK class	Keep inside	Keep outside
App	Shared services, HomeyAPI, bridge action listeners, runtime manager startup.	Mapping logic, source-specific parsing.
Driver	Pair/repair session handlers, UI data endpoints, virtual device creation.	Long-lived subscriptions, target queues.
Device	Profile loading, lifecycle registration, status, deletion cleanup.	Flow-card discovery algorithms, scheduler internals.

5. Source discovery and normalisation

5.1 Discovery order

1. Load the source device and its metadata.
2. Inspect capabilities and capability options for event-like behaviour. INTERFACE ONLY IN MVP
3. Enumerate *all* flow trigger cards and identify those accepting the selected device. Do not restrict to cards whose owner URI equals the source app — platform-generated cards may be relevant.
4. Inspect trigger arguments, dropdown and autocomplete values, and available tokens.
5. Apply a known-device recipe only when owner app, driver and event-surface fingerprint all match sufficiently.
6. Emit the normalised event catalogue consumed by the mapping UI.

5.2 Input contract

```
type InputAction =
  | 'press' | 'long_press' | 'release'
  | 'rotate_delta' | 'rotate_start' | 'rotate_stop'
  | 'selection';

interface InputEvent {
  sourceDeviceId: string;
  controlId:      string;          // stable id of the physical control
  controlLabel:   string;          // e.g. 'Dial', 'Top button'
  action:         InputAction;
  direction?:     -1 | 1;
  magnitude?:     number;          // signed, normalised
  value?:         string | number | boolean;
  provenance:     'capability' | 'flow_fixed' | 'flow_argument'
                | 'flow_token'  | 'derived';
  timestamp:      number;
}
```

5.3 Selectable catalogue

The UI consumes a stable catalogue, never live raw events.

```
interface SelectableInput {
  key:              string;          // stable binding key
  controlId:        string;
  label:            string;          // 'Dial — Turn right'
  action:           InputAction;
  direction?:       -1 | 1;
  carriesMagnitude: boolean;
  binding:          LogicalSourceBinding;
}
```

5.4 Magnitude collapse CRITICAL

Events differing *only* in magnitude are one selectable entry. A dial that exposes separate triggers or argument values for one, two, three… detents must appear as a single *Dial — Turn right*. Magnitude is preserved in the binding and forwarded at runtime; it is never a user-facing choice.

Keep multi-click count and rotary movement count semantically distinct. A Matter `count` field frequently represents wheel detents rather than repeated clicks; misreading it produces a picker full of nonsense entries and a dimmer that jumps.

5.5 Never infer unsupported gestures

- Do not offer *Long press* if only *Press* exists in the catalogue.
- Do not offer a continuous hold-ramp without a reliable release or stop signal, or repeated source events. Where absent, offer stepping instead.
- Do not derive double-press in MVP.
- When a control's semantics cannot be determined confidently, mark it unsupported rather than guessing.

6. Managed flow bridge

6.1 Purpose

Where input exists only as an external flow trigger, generate ordinary Homey Flows calling internal action cards owned by this app. Users never author these.

6.2 Bridge action cards

Card	Arguments	Use
`Receive controller event`	`controller`, `event_key`	Fixed and enum events — button presses, discrete rotation.
`Receive numeric controller event`	`controller`, `event_key`, `value`	Rotary magnitude, count, numeric delta, tokenised values.

6.3 Binding compiler

```
type LogicalSourceBinding =
  | { kind: 'direct_capability'; capabilityId: string; interpreter: ValueInterpreter }
  | { kind: 'flow_fixed';  cardId: string; cardOwnerUri: string; args: Record<string, unknown> }
  | { kind: 'flow_enum';   cardId: string; argument: string; value: unknown }
  | { kind: 'flow_range';  cardId: string; argument: string; valueRange: [number, number] }
  | { kind: 'flow_token';  cardId: string; args: Record<string, unknown>; tokenId: string };
```

6.4 Generation strategy

- Create only the variants needed by **mapped** events, never every discovered event.
- Use the fixed bridge card for button-like events; the numeric card where a token can carry the amount.
- **Range expansion:** where a numeric value is a fixed trigger argument rather than a token, compile the logical event into the necessary range of flow variants, each passing its fixed value to the numeric bridge card. This is what makes one *Dial — Turn right* entry work against an integration that models detents as separate argument values.
- Cap range expansion at a configurable ceiling (default 12 variants). Beyond it, mark the control unsupported and log — silently generating fifty flows is worse than declining.
- Keep all managed flows in a clearly named app-managed folder.
- Creation is idempotent, keyed on binding ID plus variant key.
- Persist `flowId`, logical binding ID, variant key, schema fingerprint and managed version.
- **Never overwrite a flow that appears materially user-edited.** Mark the controller for repair instead.

6.5 Reconciliation

Run on app start, controller start, repair, and relevant source or integration change. Verify existence and whether each flow is broken. Recreate missing flows *only* where the binding schema is still compatible. If the event surface changed, invalidate the binding and require remapping rather than guessing — with the one exception in §9.4.

7. Light control runtime

7.1 Targets and intents

```
type TargetSpec =
  | { kind: 'devices'; deviceIds: string[] }
  | { kind: 'zone'; zoneId: string; includeSubzones: boolean };

type LightIntent =
  | { type: 'toggle' }
  | { type: 'power';                value: boolean }
  | { type: 'brightness_delta';     delta: number }
  | { type: 'brightness_absolute';  value: number }
  | { type: 'temperature_delta';    delta: number }
  | { type: 'temperature_absolute'; value: number };
```

Zones re-resolve on device create, delete and zone-update events, so lights added to a zone are picked up without reconfiguration. Per target, inspect `onoff`, `dim` and `light_temperature` *and their capability options* — do not assume identical min, max or step semantics. Maintain a per-target support map and execute each intent only where supported.

7.2 Perceptual scale

All brightness arithmetic occurs on a perceptual axis `p ∈ [0,1]`, with device value `v = p^γ`, `γ = 2.2`. Deltas apply to `p`, then convert. Linear stepping feels violent at the bottom of the range and inert at the top.

7.3 Group power

Toggle is a group action, not independent inversion. **If any controllable target is on, turn all controllable targets off; otherwise turn all on.** This gives predictable room-level behaviour once individual lights drift out of sync.

7.4 Group brightness — relative by default

Apply the same normalised delta to every target based on that target's own current or desired level, preserving deliberate differences.

```
Before:  Ceiling 70%   Pendants 50%   Wall 30%
+5%      →
After:   Ceiling 75%   Pendants 55%   Wall 35%
```

A synchronised mode setting all compatible targets to one absolute value is available in advanced settings. It must not be the default: it destroys existing lighting composition.

7.5 Optimistic desired state

```
interface TargetRuntimeState {
  actualOn?: boolean;   actualDim?: number;   actualTemperature?: number;
  desiredOn?: boolean;  desiredDim?: number;  desiredTemperature?: number;
  lastNonZeroDim?: number;
  lastExternalUpdateAt?: number;
}
```

- Never read-modify-write per rotary tick. Initialise from current Homey state, update on event arrival, clamp, then schedule the write.
- Subscribe to target capability changes to reconcile external changes.
- Coalesce events inside a short burst window; serialise writes per target; cap write frequency to avoid saturating Zigbee, Z-Wave or cloud integrations. **Guarantee a final write after the burst ends.**
- Acceleration is optional and off by default on lossy transports — dropped detents make it fire unpredictably. Raw magnitude must remain available to the intent calculation regardless.
- Never persist transient queues. Rebuild runtime state from live device values after restart.

7.6 Supersede gate CRITICAL

STYRBAR emits a short-press event before a hold on some controls. Untreated, holding to dim also toggles the light.

When — and only when — the same `controlId` carries both a discrete and a hold mapping, delay the discrete action by `supersedeMs` (default 250). If the hold's leading event arrives inside the window, cancel the discrete action and start the ramp. Otherwise execute normally. **Controls with a single mapping incur zero added latency.** Never apply globally.

7.7 Ramp safety CRITICAL

Where a hold-ramp is offered at all (§5.5 governs when):

- Tick at 100 ms; default rate 60% of perceptual range per second.
- **Hard stop after 10 seconds. Not configurable.** Release events are routinely dropped on Zigbee and unreliable on Matter/Thread — a stuck ramp is a certainty, not a risk.
- Also stop on: any other input from the same controller, target unavailability, app shutdown.
- Cancel all ramps on `onUninit`. Never leave a light mid-ramp across a restart.

7.8 While-off policy

- *Increase brightness* while off → turn on and apply the change. (Default.)
- *Decrease brightness* while off → update desired level only. Expose "ignore" as the alternative.
- Temperature changes must not implicitly turn a light on.
- Where the resulting level would fall below the minimum, turn off rather than clamping — configurable.

7.9 Failure policy and latency

Execute target writes independently and aggregate results. One light failing must not prevent updates to the others. Record failed target IDs and the most recent error in diagnostics without surfacing routine transient failures as blocking UI. Partial capability support behaves the same way: execute on the compatible subset, disclose in the UI, never fail the whole intent.

Transport	Target press-to-light	Note
Zigbee local (STYRBAR)	< 200 ms	The reference case.
Hue Bridge (Dimmer, Tap Dial)	< 450 ms	Bridge round-trip dominates; not improvable from the app.
Matter/Thread (BILRESA)	< 600 ms, unreliable	Delivery itself is unreliable. Do not tune against this.

8. User experience

8.1 Screen A — choose remote

```
Choose remote

  Kitchen STYRBAR            IKEA Home Smart      6 controls
  Living room Tap Dial       Philips Hue          4 buttons + dial
  Bedroom Hue Dimmer         Philips Hue          4 controls
  ──────────────────────────────────────────────────────────
  Show all devices                                      ▾
```

- Rank plausible sources first; never hard-filter. Everything else stays reachable below the divider.
- **Always display the owning integration.** Identical hardware exposes different event surfaces through different pairing paths.
- The user never chooses an adapter type.
- If no usable events are found, allow selection but show "No usable remote events found" and block completion until diagnostics or repair resolves it.

8.2 Screen B — choose lights

```
Choose lights

  ● Pick lights          ○ Use a zone

  [x] Ceiling 1      [x] Ceiling 2      [x] Dining pendant
  [ ] Cabinet        [ ] Floor lamp

  3 lights selected

  Selected lights support
    On/off              3 of 3
    Brightness          3 of 3
    Colour temperature  2 of 3
```

Multi-select is a core requirement, not an advanced option. Zone selection is offered at equal prominence. The capability summary drives which function rows appear on the next screen.

8.3 Screen C — map controls

```
Kitchen Remote
Controls 3 lights                                        Edit

  ON / OFF     [ Center button – Press          ▾ ]   Test
  BRIGHTER     [ Dial – Turn right              ▾ ]   Test
  DIMMER       [ Dial – Turn left               ▾ ]   Test
  WARMER       [ Button 3 – Press               ▾ ]   Test   2 of 3
  COLDER       [ Not assigned                   ▾ ]

  + Add function                              Advanced ▸

                                                  [ Save ]
```

Function-first. Each light function has exactly one event selector. Users never configure raw events one by one outside diagnostics. Show only functions relevant to the selected targets; where support is partial, mark the row *2 of 3 lights* rather than hiding it — hiding makes the app look broken, silently skipping makes it feel broken.

Event picker

```
Increase brightness

  DIAL
    ( ) Turn right
    ( ) Turn left
  BUTTON 1
    ( ) Press
    ( ) Long press
  BUTTON 2
    ( ) Press
  ──────────────────────────
    (•) Not assigned
```

Grouped by physical control, showing only normalised selectable events. Magnitude variants are already collapsed by §5.4 and must

never surface here.

Test control HIGH VALUE

Every assigned row carries a *Test* control that executes the intent directly against the selected targets — no flow required, works before save. This is the primary defence against silent failure and the first diagnostic step when a user reports nothing happening. It also surfaces the BILRESA missing-cards condition during setup rather than three weeks later.

Conflict rule

One lighting function per normalised event in MVP. Selecting an already-assigned event prompts to replace the previous mapping. The data model permits one-to-many later; the initial UI must prevent accidental duplicates.

Add function

Category	Functions
Power	Toggle, Turn on, Turn off
Brightness	Increase, Decrease, Set brightness LATER
Temperature	Warmer, Colder, Set temperature LATER
Other	Scene and colour actions reserved

8.4 Advanced settings

Keep tuning behind the relevant mapping row or an Advanced disclosure. Do not put a generic settings form in the main flow.

Action	Settings
Increase / decrease brightness	Step and sensitivity; acceleration; behaviour while off; minimum and maximum; relative vs synchronised group mode.
Warmer / colder	Step and sensitivity; partial-target behaviour; clamp to each device's range.
Toggle / on / off	Restore-previous-brightness policy reserved for later; otherwise none.
Rotary	Use magnitude where available; burst coalescing window; never expose raw token or count variants.

8.5 Save, edit, delete

- Save creates or updates the virtual controller and its versioned profile.
- Editing reuses the same three screens with current values preselected.
- Changing the source invalidates bindings and triggers rediscovery before the mapping screen.
- Removing a target leaves mappings intact and recomputes capability availability.
- Deleting the controller removes only flows provably managed by it, plus its runtime subscriptions.

9. Persistence, health and repair

9.1 Controller profile

```
interface ControllerProfile {
  schemaVersion: number;
  enabled: boolean;
  source: {
    deviceId: string;
    ownerAppId?: string;
    driverId?: string;
    eventSurfaceFingerprint: string;
  };
  target: TargetSpec;                  // devices or zone
  mappings: MappingRule[];
  behavior: ControllerBehavior;
  managedFlows: ManagedFlowReference[];
}

interface MappingRule {
  id: string;
  function: 'toggle' | 'on' | 'off'
          | 'brightness_up' | 'brightness_down'
          | 'warmer' | 'colder';
  inputKey: string | null;             // null = not assigned
  target: TargetSpec | null;           // null = inherit; UI deferred to Phase 3
  options?: Record<string, unknown>;
}
```

Homey IDs are authoritative; names are display-only caches. Persist profiles in the virtual device store; use app-level settings only for global configuration and migrations. Every profile carries `schemaVersion` and migrates deterministically at startup.

9.2 Controller states

State	Meaning	User action
Ready	Source, mappings, managed flows and at least one target operational.	None.
Partial	Some targets unavailable or lacking a mapped capability.	Optional review.
Needs repair	Event surface changed, source deleted, or a binding cannot be safely recreated.	Open repair.
Disabled	Intentionally disabled.	Enable.

9.3 Event-surface fingerprint

Fingerprint the trigger and capability schema used at configuration time: card owner URI, card ID, argument names and types, enum and range values, relevant token IDs and types, interpreted capability metadata. This is what makes repair safe after an integration update, and it is strictly stronger than hashing the generated flow alone.

9.4 One-tap re-attach CRITICAL

> WHY THIS IS NOT OPTIONAL
>
> On BILRESA, trigger cards disappear after a Homey restart and the device must be removed and re-added — recurring, not exceptional. A generic "needs repair" state that makes the user redo the mapping every time defeats the product.

When a device appears whose owner app, driver and event-surface fingerprint match a broken controller's source, offer one-tap re-attach: rebind to the new device ID, regenerate the managed flows, **preserve every mapping and target**. Where the fingerprint has changed, fall back to §6.5 and require remapping — never guess a new binding.

9.5 Diagnostics

For troubleshooting and development, not setup. Expose source metadata, discovered normalised events, raw flow-card references, managed flow IDs and status, target capability matrix, last received event, last executed intent, recent failures, and a profile export suitable for bug reports. Never expose secrets or unrelated Homey configuration.

Structured logging keyed on controller ID and binding ID. Log every generated flow create, update and delete. Rate-limit repeated transient target errors. Never log credentials. Verbose debug enabled globally from app settings.

10. Codebase structure

```
app.ts
lib/
  homey-api-service.ts    device-catalog.ts    source-discovery-service.ts
  profiles/     controller-profile.ts  profile-repository.ts  migrations.ts
  inputs/       input-event.ts  selectable-input.ts  source-adapter.ts
                source-adapter-registry.ts  capability-source-adapter.ts
                managed-flow-source-adapter.ts  known-recipe-source-adapter.ts
                event-normalizer.ts  magnitude-collapser.ts
  mapping/      mapping-engine.ts  mapping-types.ts  action-registry.ts
                supersede-gate.ts
  outputs/      light-intent.ts  light-target-adapter.ts  target-resolver.ts
                target-state-cache.ts  command-scheduler.ts  ramp-engine.ts
  bridge/       flow-bridge-manager.ts  flow-binding-compiler.ts
                flow-card-inspector.ts  managed-flow-reconciler.ts
  runtime/      controller-runtime.ts  controller-runtime-manager.ts
                health-monitor.ts
drivers/controller/
  driver.ts  device.ts  driver.compose.json
  pair/  source.html  targets.html  mapping.html
.homeycompose/
  app.json  flow/actions/bridge_event.json  flow/actions/bridge_numeric_event.json
settings/index.html
test/  unit/  integration/  fixtures/
```

10.1 Environment

```
npm install --global homey
homey login
homey app create
homey app add-types            # if TypeScript was not selected at create
npm install homey-api
homey app run                  # live logs
homey app install              # persistent install — required for restart testing
homey app validate --level publish
```

```
{ "sdk": 3, "runtime": "nodejs", "platforms": ["local"],
  "permissions": ["homey:manager:api"] }
```

Test restart behaviour with `homey app install`, not `homey app run`. Reconciliation and re-attach are only exercised by real restarts, and they are the difference between this app working and appearing broken.

11. Testing

11.1 Unit

Module	Required cases
Event normalizer	Boolean edges; enum events; signed delta; absolute and wrapping dial; **magnitude collapse**; duplicate suppression; count-vs-detent disambiguation.
Flow binding compiler	Fixed cards; enum expansion; token forwarding; fixed numeric range expansion; expansion ceiling; deterministic fingerprints.
Mapping engine	Mapped and unmapped inputs; conflict replacement; partial target capability.
Supersede gate	Press alone; hold alone; press-then-hold inside window; press-then-hold outside window; zero added latency when only one mapping exists.
Command scheduler	Burst coalescing; rate limiting; guaranteed final write; per-target serialisation; external-state resync; perceptual curve conversion.
Ramp engine	Normal stop; missing release → 10 s hard stop; interruption by another input; cancellation on uninit.
Multi-light	Relative delta preservation; clamping; partial failures; group toggle semantics; zone re-resolution.
Migrations	Every historical schema fixture migrates without data loss.

11.2 Integration fixtures

Capture serialised device metadata and flow-card schemas for the four reference devices so discovery and compilation are testable without hardware. Keep raw fixture data separate from the normalised expected catalogue — that separation is what proves the normalizer, rather than the fixture, is doing the work.

11.3 Hardware acceptance

Device	Acceptance
STYRBAR	All exposed press and long-press controls selectable; mapped functions execute against multiple lights; **holding a control that also has a press mapping ramps without firing the press action**.
Hue Dimmer v2	Four distinct controls discovered; no unsupported hold offered; each button maps independently.
Tap Dial	Four buttons plus left/right dial; magnitude affects brightness; rapid dial motion does not flood targets and ends at a correct final value.
BILRESA	Runtime-discovered wheel semantics compile correctly; count variants appear as one directional option; **restart-induced card loss produces one-tap re-attach with mappings intact**.

11.4 Lifecycle

- Restart Homey and the app; verify controllers recover.
- Delete the source device → Needs repair, mappings retained.
- Delete one of several targets → remaining lights continue.
- Delete a managed flow → safe recreation where the schema matches.
- Modify a managed flow → app does not silently overwrite.
- Change a source integration's card schema → fingerprint mismatch handled per §9.4.
- Delete the controller → only its owned flows are cleaned up.

12. Security and robustness

- Request only required permissions. `homey:manager:api` is intentionally broad and must be documented in submission materials.
- Transmit nothing off Homey — no device inventories, configuration or event logs — absent an explicit opt-in telemetry feature.
- Never execute user-provided code or evaluate flow values as code.
- **Validate every incoming bridge argument** against an existing controller and expected binding key before acting. Generated flow arguments are user-editable and therefore untrusted.
- Never allow a generated flow to control a controller other than the one encoded in its managed binding.
- On malformed or stale events, fail closed: log and ignore rather than execute heuristically.
- Bounded queues; clean up listeners and timers on stop and delete to prevent leaks and event storms.

13. Acceptance criteria

ID	Criterion
AC-01	User selects exactly one already-paired source device.
AC-02	User selects one or more already-paired lights, or a zone.
AC-03	Mapping screen lists Toggle, Brightness ± and Temperature ± only when applicable to the selected targets.
AC-04	Each function assigns to one normalised event or Not assigned.
AC-05	**The picker never exposes raw magnitude variants as separate choices.**
AC-06	Input is received through generated bridge flows with no manual flow authoring at any point.
AC-07	Relative brightness applies to all compatible targets and preserves inter-light differences.
AC-08	Group toggle turns all off if any is on, otherwise all on.
AC-09	Partial capability or single-target failure does not block compatible targets.
AC-10	High-frequency rotary events are coalesced and rate-limited, ending at a correct final state.
AC-11	**Holding a control that also carries a press mapping does not fire the press action.**
AC-12	**Every ramp terminates within 10 seconds regardless of whether a release event arrives.**
AC-13	A control with a single mapping shows no measurable added latency versus a hand-built flow.
AC-14	On restart, controllers restore listeners and reconcile managed flows.
AC-15	A changed event surface produces Needs repair rather than a guessed binding.
AC-16	**A source whose cards vanish and reappear re-attaches in one tap with all mappings preserved.**
AC-17	Editing reopens the same configuration with existing values preselected.
AC-18	Deleting a controller cleans up only resources demonstrably owned by it.
AC-19	Test controls execute against live targets before save.
AC-20	All four reference devices validate per §11.3.

14. Implementation plan

Phase	Scope	Exit
0 — Spike	§3 in full. Both source paths proven, or the fallback ladder chosen.	Written answer on flow-write permission. Latency measured per transport.
1 — Vertical MVP	Virtual controller and persistence; three-screen pair flow; fixed and enum bindings; toggle, brightness delta, temperature delta; multi-light relative brightness; scheduler; Test controls; STYRBAR and Hue Dimmer.	A STYRBAR controls three lights end-to-end, configured by someone who has never used the app, with no visit to the Flow editor.
2 — Rotary and hardening	Numeric token bridge; range-expanded compilation; magnitude normalisation and burst coalescing; supersede gate; ramp safety; perceptual curve; health states and reconciliation; Tap Dial and BILRESA.	Holding STYRBAR's top control ramps without toggling. Rapid dial motion ends at the right level without flooding.
3 — Repair and polish	Repair flow and one-tap re-attach; diagnostics and export; fingerprints and migrations; per-action target UI; zone targets; press-to-identify accelerator; Danish localisation; App Store permission copy.	A BILRESA that loses its cards re-attaches in one tap. A Tap Dial drives three rooms from one controller.

15. Decisions locked

Decision	Setting
Controller device class	`service`, unless testing shows `remote` is materially better.
Group toggle	Any on → all off; otherwise all on.
Group brightness	Relative. Synchronised is advanced-only.
Brightness curve	Perceptual, γ = 2.2.
Increase while off	Turn on and apply.
Decrease while off	Update desired level only.
Partial capability	Apply to compatible targets; disclose in UI.
Duplicate event mappings	Disallowed in MVP; replace with confirmation.
Double press	Out of MVP.
Supersede window	250 ms, only where a control carries both discrete and hold mappings.
Ramp hard stop	10 s, not configurable.
Range expansion ceiling	12 flow variants; beyond that, mark unsupported.
Managed flow type	Ordinary Flows first. Consider Advanced Flow only if BILRESA-style fan-out becomes unwieldy.
Per-action targets	Schema in MVP, UI in Phase 3. Workaround: one controller per target group.

> IF YOU BUILD ONE THING FIRST
>
> Phase 1 without ramping and without rotary. Stepped brightness plus group toggle, across multiple lights, configured in three screens, is already better than what any of these remotes offers today — and it is the part every one of them can reliably feed.

16. Platform references

Re-check against the installed CLI and SDK version before implementation begins.

Reference	URL
Homey Web API — homey-api	athombv.github.io/node-homey-api/
Web API — Local ManagerFlow	athombv.github.io/node-homey-api/HomeyAPIV3Local.ManagerFlow.html
Apps SDK — Getting started	apps.developer.homey.app/the-basics/getting-started
Apps SDK — Custom pairing views	apps.developer.homey.app/advanced/custom-views/custom-pairing-views
Apps SDK — Flow tokens	apps.developer.homey.app/the-basics/flow/tokens
Apps SDK — Device capabilities	apps.developer.homey.app/the-basics/devices/capabilities
Apps SDK — Light best practices	apps.developer.homey.app/the-basics/devices/best-practices/lights